Distribution-Aware Distributed Database Testing

Zhou Zhou
East China Normal University, China

Si Liu
Texas A&M University, USA

Hengfeng Wei
Hunan University, China

Min Zhang
East China Normal University, China

Abstract

Distributed database management systems (DDBMSs) introduce new challenges for assessing their reliability due to distribution-specific characteristics that affect query execution and optimization. Existing testing approaches, largely designed for centralized DBMSs, often fail to explore diverse distributed execution behaviors and suffer from low executability of generated test queries, thereby limiting their effectiveness in bug detection.

We propose DAT (Distribution-Aware Testing), a novel automated testing approach for DDBMSs that systematically leverages distribution-aware information throughout the testing pipeline. DAT builds on a set of techniques that capture diverse combinations of logical schemas and data distribution strategies, and performs guided query mutation to trigger a wide range of distributed query execution behaviors and optimizations, while improving query executability via historical feedback. We implement our approach in a tool, DistRanger, and evaluate it on four widely used production DDBMSs. DistRanger uncovers 31 previously unknown bugs, including 28 distribution-specific ones whose manifestations or root causes are tied to distributed query processing and optimization, and outperforms state-of-the-art testers in bug detection.

PVLDB Reference Format:
Zhou Zhou, Si Liu, Hengfeng Wei, and Min Zhang. Distribution-Aware Distributed Database Testing. PVLDB, 1(1): XXX-XXX, 2026.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/duerwuyi/DAT.

1 Introduction

Distributed database management systems (DDBMSs) are a key component of modern cloud computing infrastructure and are widely adopted to scale storage and computation beyond the limits of centralized DBMSs [23]. Unlike their centralized counterparts, where a schema fully determines data organization, DDBMSs additionally rely on *distribution strategies* [49], such as partitioning and replication, to place data across physical nodes. This fundamental difference significantly impacts query processing. In particular, the execution of a SQL query in a DDBMS depends not only on its logical structure and semantics, but also on how the involved tables are distributed. As a result, the same query may exhibit vastly different execution behaviors, including data access patterns and communication costs, under different distribution strategies.

To achieve efficient query processing under such complex execution settings, DDBMSs implement sophisticated distributed query optimizations, e.g., shard routing [41] and co-located joins [7], to reduce cross-node communication and localize computation whenever possible. However, these optimizations, as well as distributed query processing more broadly, are highly error-prone and can result in various types of bugs and issues in practice. These include logic bugs that produce incorrect query results (e.g., the one shown in Figure 1), crashes that lead to system failures, and timeouts that prevent queries from completing within a reasonable time.

Challenges in Testing DDBMSs. Recent years have seen a surge of efforts on DBMS testing [8, 9, 31, 32, 34, 36–38, 50, 51, 53, 56, 66], which have found numerous bugs in real-world systems. However, existing testing approaches are unlikely to expose bugs like the one in Figure 1. For example, neither of the two representative state-of-the-art testers, SQLancer [47, 53] and EET [32], identified this bug in a continuous 24-hour testing campaign (see also Section 5.3). The underlying reason is that these approaches are largely designed for testing centralized DBMSs and are *agnostic to the distribution characteristics* of DDBMSs. Consequently, they exhibit fundamental limitations in two key aspects when applied to DDBMSs, namely *test diversity* and *query executability*.

First, test cases generated by existing approaches, though often complex (e.g., involving a range of SQL features and schemas), do not account for distribution-specific factors, such as distribution strategies and their interactions with query semantics. As a result, these approaches often fail to capture diverse execution behaviors unique to distributed settings. Moreover, their reliance on random testing, while effective for centralized DBMSs, often fails to trigger certain distributed query optimizations. This is because the distributed execution space is significantly larger and the conditions required to activate particular optimizations are rarely satisfied by chance. For instance, as shown in Figure 1, triggering the co-located join optimization bug in Vitess [63] requires specific structural patterns, i.e., two nested joins on a common sharding key `sdkey` (lines 12–13). This bug persisted for nearly three years across six major versions despite extensive prior testing efforts.

Second, DDBMSs often reject SQL queries due to distribution-specific constraints or unsupported features [18, 58], as such queries may require non-trivial remote execution patterns and multi-round cross-node coordination (see also Section 2.3). As a result, a large fraction of queries generated by existing approaches, though valid in centralized DBMSs, fail to execute in distributed settings, which significantly limits overall testing efficacy from the outset. For

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 1, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

```
1  CREATE TABLE t0 (sdkey INT);
2  CREATE TABLE t1 (sdkey INT);
3  /* VSchema (specify dist. strategies per table) */
4  {"sharded": true, "tables": {
5      "t0": { "column_vindexes": [{ "column": "sdkey",
           "name": "i" }] },
6      "t1": { "column_vindexes": [{ "column": "sdkey",
           "name": "i" }] } },
7    "vindexes": { "i": { "type": "xxhash" } } }
8  INSERT INTO t0 VALUES (NULL);
9  INSERT INTO t1 VALUES (1);
10
11 SELECT * FROM (t0 AS r0 LEFT OUTER JOIN
12   (t1 AS r1 JOIN t1 AS r2 ON (r1.sdkey = r2.sdkey))
13     ON (r0.sdkey = r1.sdkey));
14  --- expected result: one row returned
15  --- actual result: none
```

Figure 1: A logic bug in Vitess found by our tool, along with the test query. Vitess incorrectly returns an empty result set, while the expected result contains one row. A JSON-based VSchema specifies distribution strategies per table (lines 4–7). `sdkey` denotes the common sharding key of the two tables.

example, we observe that, out of 100K queries produced by EET, 83% are executable on PostgreSQL, whereas only 25% are executable on its distributed deployment Citus [24].

Notably, these two aspects give rise to an *inherent tension* in DDBMS testing: increasing query complexity improves test diversity, but simultaneously undermines executability. This tension is largely absent in centralized DBMS testing and poses a fundamental challenge for testing DDBMSs.

Our Approach. We introduce DAT (Distribution-Aware Testing), a novel automated testing approach for DDBMSs. It effectively exposes bugs arising from distributed query processing and optimization that are unlikely to be uncovered by existing centralized testing techniques. Our key insight is that distribution-aware information, i.e., data distribution strategies and their interaction with query semantics, can be systematically leveraged throughout the testing pipeline to address the aforementioned challenges.

DAT begins by constructing a *Schema-Distribution Matrix* (SDM), which captures a wide range of combinations across diverse logical schemas and data distribution strategies. This provides a structured yet expressive exploration space for generating varied DDBMS instances for testing. In particular, this design offers two key properties: (i) column-wise, tables derived from the same schema can be interchanged to explore different distribution strategies without violating SQL syntactic validity; and (ii) row-wise, tables instantiated under aligned distribution strategies naturally satisfy the conditions required by certain distributed query optimizations.

These properties enable effective test query generation over SDM, which DAT achieves through a generic mutator $\mu = (\rho, \Delta)$. Specifically, in addition to column-wise and row-wise table substitutions via ρ, it incorporates query transformations encoded in Δ, tailored to specific distributed query optimizations, such as shard routing and co-located joins. By applying sequences of mutations using μ, DAT-generated queries can trigger a range of distributed execution behaviors and optimizations.

To improve query executability, DAT incorporates a feedback mechanism during query mutation. By leveraging the history of previously executed and rejected queries in the DDBMS under test, it learns patterns of unsupported query structures under specific distribution strategies, which are encoded as *feature paths*, and guides subsequent generation toward diverse yet executable queries. This feedback loop, in turn, effectively mitigates the tension between test diversity and query executability.

We implement DAT in a tool called DistRanger. It currently employs differential testing as the primary test oracle, i.e., identifying discrepancies between the execution results of the same query under centralized and distributed deployments of the same DBMS; any discrepancy indicates potential issues specific to the distributed layer. However, DAT can naturally incorporate other types of oracles, e.g., metamorphic ones [32, 46].

In addition, DAT can seamlessly integrate advances in query and schema generation into base query generation prior to mutation and SDM construction, respectively. Finally, the mutator μ is general and can be instantiated to incorporate additional distributed query optimizations beyond our current focus on shard routing and co-located joins. Together, these features make DAT a general testing framework for DDBMSs. Sections 6 and 7 provide a detailed discussion of these aspects.

Contributions. Overall, we make the following contributions.

- Conceptually, we propose DAT, a novel approach for testing DDBMSs, a problem that presents fundamental challenges beyond centralized DBMS testing, by leveraging distribution-aware information throughout the testing pipeline.
- Technically, we develop a set of techniques that generate diverse test cases, including DDBMS instances and distributed queries, improve query executability, and mitigate the inherent tension between diversity and executability.
- Practically, we realize DAT in an automated tester, DistRanger, and assess it on four production DDBMSs: Citus, Vitess, Apache ShardingSphere, and ClickHouse. It uncovers 31 previously unknown bugs, 28 of which are specific to distributed query processing and optimization. Compared to the state-of-the-art, DistRanger demonstrates superior effectiveness in bug detection.

We have open-sourced DistRanger to facilitate further research on DDBMS testing and related applications.

2 Background

2.1 Distribution Strategies

In a centralized DBMS, a schema defines the logical organization of data as a set of relations (or tables). Yet, in a DDBMS, the schema alone does not determine data placement. The system must additionally define *distribution strategies*, which specify how data is physically partitioned for scalability or replicated for availability across nodes [49]. These strategies are typically defined *per table*.

In practice, DDBMSs often use different terminology for conceptually similar distribution strategies; Table 1 summarizes the terminology across four representative systems. To avoid ambiguity, we adopt the terminology of Citus [24] in this paper.

Local tables store all data on a single node, allowing queries over them to be executed without cross-node communication.

Table 1: Terminology for distribution strategies across four representative DDBMSs. Each row corresponds to a distribution strategy. This paper adopts the terminology of Citus.

Citus	ShardingSphere	Vitess	ClickHouse
Local [16]	Single [5]	Unsharded [61]	MergeTree [20]
Reference [16]	Broadcast [3]	Reference [59]	Replicated [21]
Distributed [16]	Sharding [4]	Sharded [60]	Distributed [19]
Co-located [15]	Binding [2]	Same Vindex [57]	N/A

Reference tables store a full copy of the table on each node, either via explicit replication (e.g., in Vitess) or router-level broadcast (e.g., in ShardingSphere). This enables local access during distributed query execution.

Distributed tables are horizontally partitioned across nodes based on a sharding key and sharding method. Querying such tables requires accessing multiple nodes and merging partial results.

Co-located tables are distributed tables with aligned partitioning schemes, such that rows sharing the same sharding key reside on the same node. This enables efficient joins on the sharding key, commonly referred to as *co-located joins* [7].

Different DDBMSs often specify these distribution strategies using different languages, e.g., SQL or SQL-like DDL (Data Definition Language) statements in Citus, ShardingSphere, and ClickHouse, and a JSON-based schema in Vitess (Figure 1). Note that DDBMSs may impose restrictions on certain strategies (e.g., in Citus, local tables exist only on the coordinator node), and some systems provide additional system-specific variants [44]. A table may also simultaneously exhibit both partitioning and replication (e.g., replicated shards); yet, such cases can still be expressed using these strategies.

2.2 Distributed Query Optimization

In DDBMSs, a query is processed by decomposing it into fragments, executing them across nodes, and coordinating multiple rounds to produce the final result. In this process, *distributed query optimization* plays a critical role in system performance by determining execution strategies that minimize cross-node communication overhead and localize computation whenever possible.

Below, we introduce two representative distributed query optimizations, *shard routing* and *co-located joins*, which are widely supported as first-class optimizations in existing DDBMSs [7, 14, 62] and also serve as running examples throughout the paper.

Shard Routing. Scanning a distributed table often requires accessing all of its shards. However, when a query includes a predicate on the sharding key, execution can be restricted to the shards that satisfy the predicate. This optimization is known as *shard routing* (or shard pruning) [7, 41].

Example 2.1. In Figure 2(a) and (b), the table `t2` is sharded by `t2.sdkey mod 2`. As a result, querying `t2` typically requires accessing both nodes, e.g., by `SELECT * FROM t2 WHERE t2.c0 = 140`, as shown in Figure 2(a). However, when the `WHERE` clause includes the predicate `t2.sdkey = 1`, the DDBMS can leverage the sharding strategy of `t2` to route the query to Node 1 only, as shown in Figure 2(b).

Co-located Joins. A join between two distributed tables often requires cross-shard communication. However, if the tables are co-located, a join on the sharding key can be executed locally, as rows with the same sharding key are placed on the same node. This optimization is known as a *co-located join*.

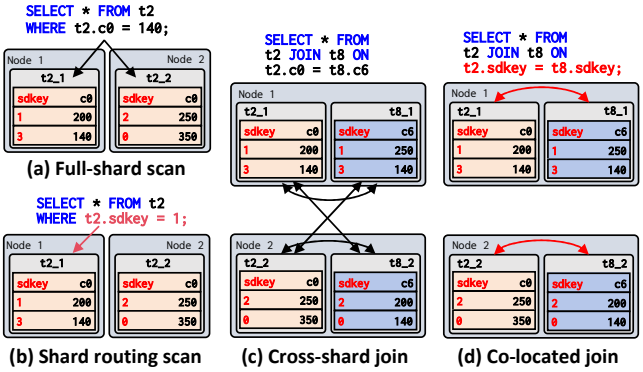

Figure 2: An illustration of the two distributed query optimizations: shard routing (a vs. b) and co-located joins (c vs. d). In each case, both tables are sharded by `sdkey`.

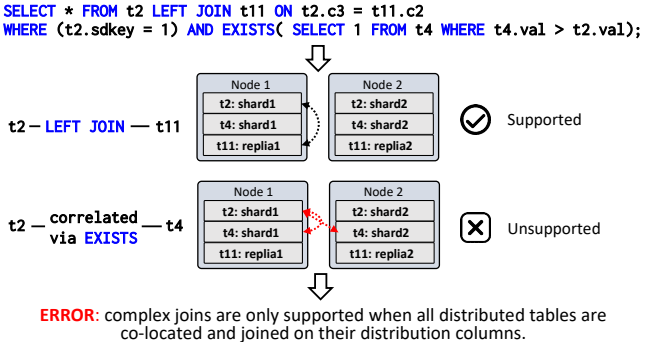

Figure 3: An example of a SQL query rejected by Citus (v13.0).

Example 2.2. In Figure 2(c) and (d), the tables `t2` and `t8` are co-located on `sdkey`, meaning that rows with the same `sdkey` reside on the same node. In Figure 2(d), the join condition `t2.sdkey = t8.sdkey` allows each shard of `t2` to join only with the corresponding local shard of `t8`, without accessing data from remote nodes. In contrast, in Figure 2(c), the join is on `c0` and `c6` rather than the sharding key. As a result, each shard of `t2` may need to join with all shards of `t8`, potentially incurring high communication overhead.

In addition to shard routing and co-located joins, DDBMSs may implement other distributed query optimizations. Nevertheless, a common theme is that these optimizations leverage data distribution strategies to reduce query execution cost in distributed settings.

2.3 Unsupported SQL Features in DDBMSs

Although distributed query optimization improves execution efficiency, DDBMSs often conservatively reject complex SQL queries at planning time and return errors indicating *unsupported features* [18, 58]. This is because processing such queries may introduce non-trivial remote execution patterns, and some executions may require multiple rounds of coordination across nodes, which fall outside the system's design assumptions.

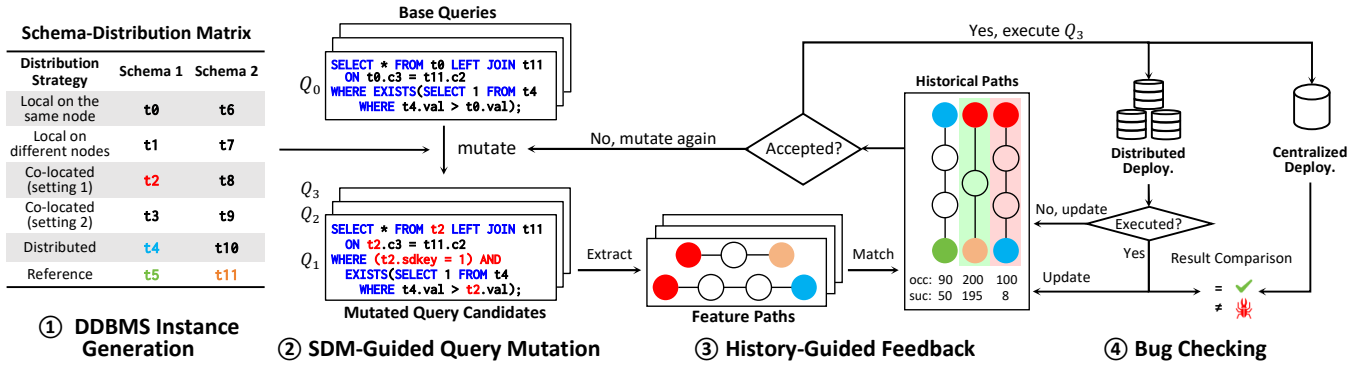

Figure 4: Workflow of our DAT approach.

Note that the set of unsupported SQL features may vary across DDBMSs and even across different versions of the same system. Nevertheless, these limitations are commonly rooted in mismatches between query semantics and data distribution constraints.

The following illustrates a SQL query that is not executable in Citus (v13.0) and serves as a running example throughout the paper.

Example 2.3. As shown in Figure 3, `t2` and `t4` are two distributed tables, while `t11` is a reference table. The query involves a shard-routing predicate `t2.sdkey = 1` that restricts access to a shard of `t2`, as well as two types of interactions between tables. The first arises from `t2 LEFT JOIN t11`, which is supported by Citus. The second is an `EXISTS`-based correlation between `t4` and `t2`. Despite shard routing on `t2`, the correlation with `t4` still induces a cross-shard semi-join, which is not supported. As a result, the planner fails to produce a query plan for the entire query, which is ultimately rejected by Citus with an error.

Query executability poses a major obstacle to DDBMS testing: a high rejection rate of generated queries prevents effective exploration of system behavior, thereby limiting bug detection.

3 Overview of DAT

This section provides an overview of DAT (Distribution-Aware Testing), an automated testing approach for DDBMSs. DAT is unique in its ability to generate diverse yet executable queries that can effectively exercise a wide range of distributed execution behaviors and optimizations, thereby uncovering issues in the distributed layer that do not arise in centralized DBMSs.

Figure 4 shows DAT's workflow, consisting of four main steps.

① **DDBMS Instance Generation.** DAT starts by generating a set of logical schemas, including tables and their relationships (e.g., via foreign keys), and populating them with random data. It then generates a set of distribution strategies, each with multiple instances depending on how data is placed. Together, the schemas and distribution strategies form the *schema–distribution matrix* (SDM), which provides a structured yet expressive space for generating diverse DDBMS instances and guiding query mutation in Step ②.

Figure 4 shows an example SDM containing two logical schemas and six distribution strategies, including randomly distributed tables, reference tables, two types of co-location settings, and tables localized on a single node or across different nodes. Overall, this yields 12 distributed table instances.

② **SDM-Guided Query Mutation.** Given a DDBMS instance, DAT generates a base query and applies mutations to diversify it, aiming to trigger a range of distributed execution behaviors and optimizations. In particular, these mutations are guided by the SDM, e.g., by replacing a table with another one under a different distribution strategy to explicitly trigger the corresponding optimizations.

Figure 4 shows an example of triggering the shard-routing optimization. Specifically, we select a table from the base query Q_0, namely the local table `t0`, on which shard routing cannot be directly applied. We then replace `t0` with another table from the same column of the SDM, namely `t2`, and insert a shard-routing predicate `t2.sdkey = 1` into the `WHERE` clause. This mutation increases the likelihood of triggering shard routing in the DDBMS under test.

Additional mutation types are described in Section 4.2.

③ **History-Guided Feedback.** Mutated queries may be rejected by a DDBMS due to unsupported features (Section 2.3), which in turn reduces overall testing efficacy. To address this, DAT incorporates a feedback mechanism that interacts with Step ②, leveraging the history of previously executed and rejected queries by the DDBMS to guide generation toward executable queries.

Specifically, this mechanism extracts structural features from a mutated query, e.g., join types and correlations, and encodes them as *feature paths* (intuitively, how tables are related through SQL operations). These paths are then matched against historical paths, which also maintain executability statistics (e.g., their frequency of occurrence and successful execution counts).

As shown in Figure 4, the mechanism extracts structural features from a mutated query Q_1, such as the correlation between `t2` and `t4` and the presence of the `t2 LEFT JOIN t11` construct, and encodes them as two feature paths. DAT rejects this candidate because one path matches a frequently rejected pattern in the historical record (in red), despite another having a high success rate (in green). As mutation proceeds iteratively, a subsequent candidate Q_3 that passes this check is submitted to the DDBMS for execution.

④ **Bug Checking.** In this step, DAT executes the mutated query against the DDBMS under test to detect bugs. As shown in Figure 4, it employs differential testing to identify discrepancies between the execution results of the same query (e.g., Q_3) under centralized and distributed deployments of the same system; any mismatch indicates potential issues specific to the distributed layer. DAT also naturally supports other test oracles; see Sections 5.2 and 6.

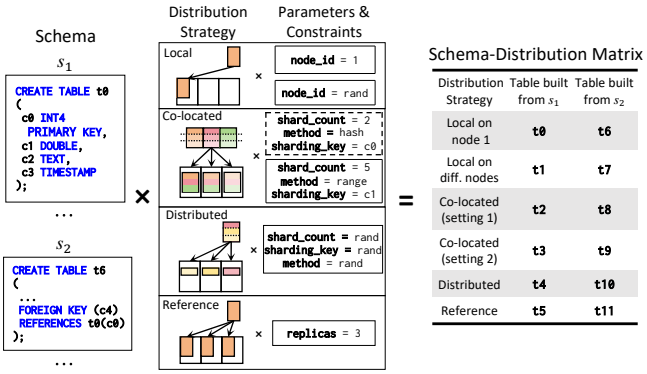

Figure 5: An illustration of how an SDM is constructed.

Note that, regardless of whether a query is successfully executed, this information is fed back to Step ③ to guide future mutations.

The above steps illustrate one iteration of our testing process, which can be repeated. In subsequent iterations, we can either continue from Step ② with a new base query or restart from Step ① to explore a new DDBMS instance with different SDMs. Step ③ persists throughout the entire testing process, recording information about all previously executed or attempted queries.

DAT leverages recent advances [31, 54] in centralized DBMS testing to produce syntactically valid SQL queries and diverse schemas; these are not part of our contributions. The next section presents DAT's core techniques tailored for testing DDBMSs.

4 Core Techniques in DAT

4.1 DDBMS Instance Generation

When generating a database instance for testing, unlike in the centralized setting where only the logical schema is considered, the distributed setting must additionally account for how tables are partitioned or replicated across physical nodes. To this end, we organize table instances in a DDBMS using the *Schema-Distribution Matrix* (SDM), denoted by $\mathcal{M}$. Let $\mathcal{S}$ be a set of logical schemas and $\mathcal{D}$ be a set of distribution strategies under consideration, with $|\mathcal{S}| = n$ and $|\mathcal{D}| = m$. Then $\mathcal{M}$ is an $m \times n$ matrix indexed by $\mathcal{D} \times \mathcal{S}$, where each entry $\mathcal{M}[d, s]$ denotes the table instance obtained by instantiating schema $s \in \mathcal{S}$ under distribution strategy $d \in \mathcal{D}$ and populating it with randomly generated data.

Figure 5 illustrates the construction of the SDM shown in Figure 4, which consists of two schemas and six distribution strategies. First, we generate a set of schemas, e.g., s_1, which specifies all information required to define a table in a centralized setting, including its attributes, data types, and integrity constraints. Next, each schema, instantiated with random data, is combined with a distribution strategy to form a distributed table instance. For example, s_1 is paired with six different distribution strategies to generate the table instances `t0` through `t5`.

How are these distribution strategies generated? By default, we randomly configure their parameters, such as the number of shards, the sharding method, and the sharding key. This yields a distributed table instance in the "Distributed" row of the SDM, e.g., `t4`.

While random generation is generally effective for exploring the space of distribution configurations, it may be insufficient for deliberately triggering certain distributed query execution logic and optimizations, which often require multiple distributed tables to satisfy alignment conditions. For example, co-located joins require participating tables to share the same sharding parameters. To this end, DAT introduces *constraints* on distribution parameters.

Example 4.1. Consider the "Co-located" case, specifically the dashed box, in Figure 5. DAT defines a parameter setting for co-located tables by constraining the shard count (2), the sharding method (hash), and the sharding key (`c0`). It then instantiates multiple tables under this setting. As a result, table instances in the same row "Co-located (setting 1)", e.g., `t2` and `t8`, are naturally aligned and thus co-located, as they share the same sharding parameter setting, even though they are instantiated from different schemas.

When generating an SDM, although the types of distribution strategies (Table 1) are relatively limited compared to schemas, we consider a larger number of their instantiations to enable diverse distributed execution behaviors and optimizations. The exact number $|\mathcal{D}|$ is determined by the testing goal and the alignment conditions required to trigger specific optimizations, and may vary across DDBMSs due to differences in their supported strategies.

Example 4.2. As shown in Figure 5, DAT introduces two parameter constraints for local tables, and similarly for co-located tables. This results in two distinct rows in each case. For example, local tables may be placed on a fixed node (both `t0` and `t6` on Node 1) or on different nodes (for `t1` and `t7`). Compared to "Co-located (setting 1)", `t3` and `t9` are co-located tables instantiated under a different parameter setting (five shards, range sharding on the key `c1`).

Note that in an SDM, tables share *the same schema column-wise*. Replacing one with the other preserves SQL syntactic validity while potentially inducing different distributed execution behaviors. Tables share *the same distribution strategy row-wise*, enabling preconditions required by certain optimizations (e.g., co-located joins). Overall, the SDM provides a strong basis not only for generating diverse DDBMS instances, but also for guiding query mutation to explore a broad range of distributed behaviors, as we will see next.

4.2 SDM-Guided Query Mutation

With the SDM in place, the next step is to generate test SQL queries. We start from base queries that are either generated by existing tools [31, 54] or observed to be executable during testing. For the former, we initialize them to access only local tables on the same node, i.e., from the "Local on the same node" row of the SDM. Such queries are unlikely to be rejected because this setting closely resembles a centralized DBMS (except that communication occurs only between that node and the coordinator).

Given a base query, DAT proceeds by applying a series of mutations to it, aimed at triggering diverse distributed query executions and optimizations. Underlying this process is a generic mutator $\mu = (\rho, \Delta)$, where ρ maps each table instance $\mathcal{M}[d, s]$ in the base query to another (or the same) table instance $\mathcal{M}[d', s']$ in the SDM, and Δ denotes a set of additional (optional) transformations to the base query tailored to specific distributed query optimizations.

In the following, we present three instances of μ, with corresponding examples shown in Figure 6. The first serves as a general mutation for exploring diverse distributed behaviors, while the

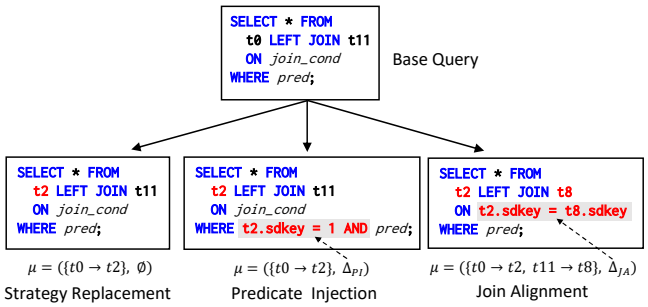

Figure 6: Examples of three types of query mutations, along with corresponding ρ and Δ specifications.

remaining two target specific distributed query optimizations. We assume that the base query is executable in the DDBMS under test.

Strategy Replacement (SR). In this mutation, ρ replaces tables in a query with other tables from the same column of the SDM (i.e., $s = s'$) but with different distribution strategies (i.e., $d \neq d'$). Since the new tables share the same schema as the original ones, the mutated query remains syntactically valid. Oftentimes, changing the distribution strategy of even a single table can already alter distributed query execution behavior. For example, ρ can be used to trigger different join optimizations across table types, e.g., between a reference table and a distributed table. Note that SR does not involve additional transformations to the base query, i.e., $\Delta_{SR} = \emptyset$.

Example 4.3. In Figure 6 (left branch), ρ replaces `t0` with `t2`. Before the mutation, `t0` and (a copy of) the reference table `t11` can be joined locally on the same node. After the mutation, the join structure is preserved, but its execution semantics change, as `t0` is replaced by the distributed table `t2` (recall from Section 2.1 that co-located tables are a special case of distributed tables). Consequently, the join may no longer be executed locally on the same node.

Predicate Injection (PI). This mutation is designed to trigger shard routing, a distributed query optimization that limits query execution to a subset of relevant shards rather than all shards. In addition to introducing at least one distributed table (via ρ guided by the SDM), this mutation applies Δ_{PI} which inserts a predicate into the **WHERE** clause of the base query. In particular, the predicate must be defined on the sharding key of the distributed table and inserted into the filter condition associated with that table.

Example 4.4. In Figure 6 (middle branch), after ρ replaces `t0` with `t2`, Δ_{PI} inserts the predicate `t2.sdkey = 1` into the **WHERE** clause, making the sharding key of `t2` explicitly appear in the filter condition. When processing this query, the optimizer can infer that only the shards containing rows with `t2.sdkey = 1` are relevant.

Join Alignment (JA). This mutation targets co-located joins, a query optimization that executes joins locally between distributed tables without cross-shard communication. Δ_{JA} applies only when all table instances participating in a **JOIN** are already co-located after applying ρ (based on the SDM). It then rewrites the join conditions so that joins between the participating tables are explicitly expressed as equalities on their common sharding keys.

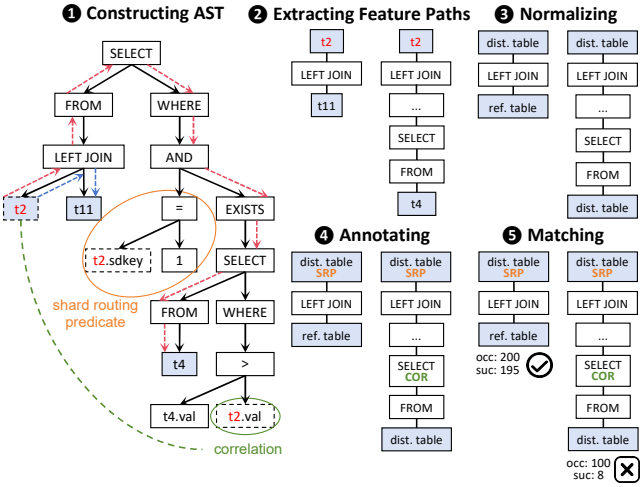

Figure 7: An illustration of the history-guided feedback mechanism using the SQL query in Figure 3.

Example 4.5. In Figure 6 (right branch), after applying ρ, `t0` and `t11` are replaced with `t2` and `t8`, respectively. Δ_{JA} then rewrites the **ON** predicate as `t2.sdkey = t8.sdkey`, where `t2` and `t8` are co-located on the sharding key as determined during SDM construction. Consequently, the join is transformed into a form that is likely to be executed as a co-located join.

4.3 History-Guided Feedback

While the mutation diversifies generated queries, many may be rejected by the DDBMS due to unsupported SQL features (Section 2.3), reducing overall testing efficacy. To address this, we introduce a feedback mechanism that guides the generation toward executable queries by leveraging historical successful and failed cases.

Figure 7 illustrates our history-guided feedback mechanism using the SQL query in Figure 3. It begins by parsing the *abstract syntax tree* (AST) [1] of a mutated query candidate, which provides a structured representation of the query's syntactic constructs. Formally, the AST of a SQL query Q is defined as $A(Q) = (V, E)$, where V is the set of AST nodes representing syntactic constructs in Q, including statement constructs (e.g., **SELECT**), table-reference constructs (e.g., `t2`), relational operator constructs (e.g., **LEFT JOIN**), and expression constructs (e.g., the **AND** predicate); $E \subseteq V \times V$ is the set of directed edges capturing parent–child relationships between nodes. The AST corresponding to the query in Figure 3 is shown in Step ❶, which can be constructed using existing approaches [31, 47].

We next describe the remaining four core steps, which determine whether a mutated query candidate qualifies as a test query.

❷ Extracting Feature Paths. We aim to avoid generating mutated queries that are similar to those previously rejected. A key challenge, therefore, is how to represent such similarity.

Our insight is that if a base query is executable but a mutated query is not, the cause of rejection likely stems from unsupported features introduced by the replaced tables, as the mutation preserves the schema but alters their distribution strategies, along with the associated transformations (as defined in μ). To capture this, we define *feature paths*. Intuitively, a feature path captures

the structural relationship between two table nodes in the AST, encoding the sequence of relational operators and predicates that connect them, and thus characterizes how their distribution strategies jointly affect the executability of the query.

Feature paths are extracted by starting from each mutated table node and traversing the AST, viewed as an undirected graph, to other table nodes. All intermediate nodes along such paths correspond to SQL constructs. As the AST is a tree, the path between any two table nodes is unique. Note that paths between unchanged tables are not extracted, as they are assumed to remain executable given that they already appear in the original executable query. As shown in Figure 7 (❷), two feature paths are extracted from the AST by traversing the blue and red paths, respectively, both starting from the mutated table `t2`.

❸ **Normalization.** Queries that exhibit similar distributed execution behaviors may nonetheless produce different feature paths, because the above representation retains concrete table identities. For example, the execution of `t2` **`LEFT JOIN`** `t11` may be similar to that of `t9` **`LEFT JOIN`** `t5` in the DDBMS, as the involved tables share the same distribution strategies (i.e., a distributed table joined with a reference table; see Figure 5), yet are treated as distinct feature paths due to their different names. This introduces unnecessary variability, which inflates the feature path space, leads to a lower matching rate with historical patterns in Step ❺, and ultimately weakens the effectiveness of the feedback guidance.

To address this issue, we normalize feature paths based on the *effective distribution roles* of table references, rather than their concrete names. The effective role of a table reference is the distribution strategy it exhibits in the given query context, as determined by its SDM configuration and the surrounding SQL structure. For example, `t2` behaves as a distributed table, as shown in Figure 7 (❸), despite originating from a co-located table in Figure 5, as it does not participate in any co-located join.

After normalization, two feature paths are considered equivalent if they share the same sequence of non-table SQL constructs and their endpoint table references have the same effective distribution roles. This yields a canonical representation, enabling matching across queries with similar distributed execution behaviors.

❹ **Annotation.** Another limitation of original feature paths is that they fail to capture execution-relevant information outside the path. For example, a shard-routing predicate (`t2.sdkey = 1`) may restrict access to `t2` to only a subset of its shards during distributed execution, rather than all shards. Such information may significantly affect execution behavior (e.g., by reducing cross-shard communication), but is not reflected in the feature path itself.

To address this issue, we augment feature paths with *node-level annotations* to capture execution semantics beyond structural information, thereby enabling more accurate matching. In our current design, we consider two annotation families most relevant to distributed execution behaviors in DDBMSs: *optimization annotations* and *correlation annotations*.

Optimization Annotations. These annotations capture distributed optimizations associated with tables or operators in a query, such as shard routing and co-located joins, which are frequently triggered by our mutations. For example, for shard routing, we record routing predicates and annotate the corresponding table node with "SRP". For example, the starting node of each path in Figure 7(❹) is annotated to indicate that a shard-routing predicate is applied to `t2`. Similarly, for co-located joins, normalizing participating tables into co-located tables is insufficient, as multiple **`JOIN`**s may appear along a path. Therefore, we annotate the specific **`JOIN`** nodes that perform co-located joins with "CLJ".

Correlation Annotations. A correlation occurs when a subquery references a table in an outer query block. For example, `t2` is referenced in an **`EXISTS`** subquery (via the predicate `t4.val > t2.val`), creating a dependency between the outer query and the subquery. Such dependencies may introduce additional execution and communication overhead in DDBMSs, e.g., when subquery evaluation depends on values from the outer query; moreover, they may require complex distributed semi-join or anti-join execution across shards, potentially causing an otherwise executable query to be rejected. Accordingly, correlation annotations capture such cross-block dependencies by annotating the corresponding nodes with "COR", e.g., the **`SELECT`** node in Figure 7(❹), so that correlated paths can be distinguished during matching.

Note that, following Step ❸, feature path equivalence is then redefined by incorporating these annotations.

❺ **Matching against Historical Paths.** DAT maintains a path set S_H that stores feature paths extracted from all previously executed test queries, together with their execution statistics. Each element in S_H is represented as a tuple (p, occ, suc), where p is a feature path obtained after normalization and annotation, occ is its occurrence count, and suc is its success count. Two paths are considered a *match* if they are equivalent, independent of their associated statistics.

Let Q be a mutated query candidate, and let $\mathcal{P}(Q)$ denote its set of (normalized and annotated) feature paths. For each $p \in \mathcal{P}(Q)$, if there exists an entry $s = (p', occ, suc) \in S_H$ such that p matches p',

$$occ(s) \geq k \quad \text{and} \quad \frac{suc(s)}{occ(s)} \leq \theta$$

then Q is filtered out by DAT; otherwise, Q is submitted to the DDBMS for execution. Here, k is the minimum occurrence threshold and θ is the success-rate threshold. In practice, we choose a sufficiently large k (e.g., 10) to ensure that decisions are based on reliable historical evidence. Moreover, some unsupported features may not always lead to query rejection, e.g., due to optimizer pruning. Therefore, a path is considered unsupported only when its empirical success rate is consistently below θ. In our experiments, we set $\theta = 10\%$, and the rationale is provided in Section 5.4.2.

As shown in Figure 7(❺), two feature paths are matched in the history. One has a high empirical success rate (97.5%), while the other has a very low success rate (8%) due to the unsupported feature shown in Figure 3. Therefore, this candidate query is filtered out by DAT prior to its execution in the DDBMS.

5 Evaluation

We implement our DAT approach as a tool called DistRanger. It builds on the codebase of DynSQL [31], which provides a basis for generating syntactically complex and valid SQL queries that can be executed on centralized DBMSs,[1] as well as diverse database

[1]These queries involve common SQL constructs, e.g., DQL and DML, and advanced features, including subqueries, aggregation functions, and window functions.

Table 2: Summary of the tested DDBMSs.

DDBMS	Architecture	Base DBMS	GitHub Stars	LoC	Initial Release	Tested Version
Citus	extension	PostgreSQL	12.5K	350K	2012	v12.5 w/ PostgreSQL 16.2; v13.2 w/ PostgreSQL 17.2; v14.0 w/ PostgreSQL 18.2
ShardingSphere	middleware	PostgreSQL	20.7K	790K	2016	v5.5.2 w/ PostgreSQL 17.0
Vitess	cloud-native	MySQL	20.9K	210K	2010	v24.0.0 w/ MySQL 8.4.8
ClickHouse	cloud-native	–	47.1K	1.1M	2012	v25.12.4.35

schemas. The overall codebase consists of 17 KLOC in C++, including 1.6 KLOC for the query mutator and the feedback mechanism, 1.9 KLOC for DDBMS instance generation (i.e., SDMs) and bug checking, and 5.7 KLOC for supporting multiple DDBMSs.

We conduct an extensive evaluation of DistRanger and compare it against state-of-the-art testers for DDBMS bug detection. We aim to address the following questions:

Q1. Can DistRanger uncover previously unknown bugs in production DDBMSs (Section 5.2)?
Q2. Does DistRanger outperform the state-of-the-art in generating high-quality queries and identifying bugs (Section 5.3)?
Q3. How does each of our key design choices contribute to DistRanger's effectiveness gains (Section 5.4)?

5.1 Experimental Setup

We evaluate recent releases of four widely used production DDBMSs of different kinds. Citus [24] extends PostgreSQL into a distributed database, enabling horizontal scaling across multiple nodes. Apache ShardingSphere [35] operates as a database middleware layer (e.g., on top of PostgreSQL), aiming to maximize the computing capabilities of existing databases. Vitess [63] is a cloud-native distributed database system built around MySQL, supporting virtually unlimited scalability through generalized sharding. ClickHouse [22] is a standalone, columnar DBMS for online analytical processing (OLAP), enabling real-time analytical queries over large datasets. Table 2 summarizes their key characteristics, including architectures, popularity, codebase sizes, and tested versions.

We deploy each DDBMS cluster using both Docker Compose and Kubernetes. For Citus and ShardingSphere, each cluster consists of one coordinator node and five data nodes. For Vitess, we employ five shard nodes and two primary nodes. For ClickHouse, we deploy two database nodes and three keeper nodes. For differential testing, we construct, for each distributed deployment, a corresponding centralized deployment on a single node. In particular, for Citus, ShardingSphere, and Vitess, the centralized deployment corresponds to their underlying base DBMSs (see Table 2).

For each SDM, we randomly generate 4–9 schemas and 4–6 distribution strategies of different types. Each generated table contains 40–120 rows chosen uniformly at random. We regenerate the SDM after every 1K generated queries, and each query is mutated for up to 10 times. While larger parameter settings (e.g., more schemas or mutation times) may expose additional bugs, the above settings already suffice to uncover many bugs in practice, as we will see.

All experiments were conducted on a 64-bit Ubuntu 22.04 machine with an AMD Ryzen 5975WX (32 cores) and 256 GB RAM. Each data point is averaged over five runs.

Table 3: Summary of the bugs detected by DistRanger, with distribution-specific bugs shown in parentheses.

DDBMS	#detected	#confirmed	#fixed
Citus	8 (8)	6 (6)	6 (6)
ShardingSphere	9 (7)	7 (5)	4 (2)
Vitess	12 (12)	3 (3)	0 (0)
ClickHouse	2 (1)	0 (0)	0 (0)
Total	31 (28)	16 (14)	10 (8)

Table 4: Distribution of the detected bugs by type.

Bug Type	Citus	ShardingSphere	Vitess	ClickHouse	Total (31)
Logic	5	4	6	1	16
Crash	3	0	2	0	5
Error	0	3	3	0	6
Exception	0	1	0	1	2
Timeout	0	1	1	0	2

5.2 Uncovering New Bugs

Overall, we reported 31 previously unknown bugs across the four DDBMSs, as summarized in Table 3. A complete list of these bugs, along with detailed descriptions, is provided in the Appendix [68]. Among them, 28 are distribution-specific, whose manifestations or root causes are tied to distribution-specific mechanisms across the query processing pipeline, including rewriting, planning, optimization, and execution. This demonstrates DistRanger's effectiveness in triggering distribution-specific bugs beyond those typically observed in centralized DBMSs. The remaining three bugs include one timestamp format inconsistency and two SQL parser issues. Notably, some of these bugs remained undetected for years, e.g., half of the bugs in Citus went undetected for over four years.

As of this writing, 16 bugs have been confirmed by the developers, with 10 already fixed in recent releases. Due to slower responses from the Vitess and ClickHouse teams, many bugs are still under investigation and have yet to be confirmed. Nevertheless, the confirmed cases increase our confidence that the remaining ones are also genuine. Moreover, our test oracles are highly unlikely to produce false positives, as they rely on clear signals such as crashes, timeouts, errors, unexpected exceptions, and discrepancies in the results of the same query under centralized and distributed executions of the *same* system. Table 4 shows the distribution of the detected bugs by type: half are logic bugs causing incorrect query results, which are uncovered via differential testing; the rest are identified via log inspection and exception monitoring.

Interesting Bugs. We presented a representative bug in Section 1. Below, we present two additional bugs uncovered by DistRanger.

Table 5: Bugs detected over 24 hours of testing. SQLancer's current implementation only supports testing Citus and ClickHouse.

DDBMS	DistRanger		SQLancer		EET		DistRanger$_{!MT}$		DistRanger$_{!FB}$	
	total	dist-specific	total	dist-specific	total	dist-specific	total	dist-specific	total	dist-specific
Citus	3	3	0	0	0	0	3	3	1	1
ClickHouse	2	1	0	0	1	1	2	1	2	1
ShardingSphere	4	2	–	–	2	1	2	0	3	1
Vitess	7	7	–	–	2	2	6	6	5	5
Total	16	13	0	0	6	5	13	10	11	8

```
CREATE TABLE t1 (vkey INT4);
CREATE TABLE t2 (vkey INT4);
INSERT INTO t2 (vkey) values (5);
SELECT create_distributed_table('t1', 'vkey');    --- t1's DS
SELECT create_reference_table('t2');              --- t2's DS

SELECT * FROM (t2 FULL OUTER JOIN t1 ON (t2.vkey = t1.vkey))
WHERE NOT((85) IN (SELECT 1 FROM t2));
--- expected result: 1 row returned
--- actual result: 32 rows returned 🐞
```

Figure 8: A logic bug found in Citus: `FULL OUTER JOIN` with a reference table yields 32 duplicate rows.

```
SET DEFAULT SINGLE TABLE STORAGE UNIT = ds_0;     --- t0's DS
CREATE TABLE t0 (c2 TEXT,c6 TEXT);
SET DEFAULT SINGLE TABLE STORAGE UNIT = ds_1;     --- t1's DS
CREATE TABLE t1 (vkey INT4);
CREATE BROADCAST TABLE RULE t2;                   --- t2's DS
CREATE TABLE t2 (vkey INT4);

SELECT * FROM t0 AS ref_0
WHERE (ref_0.c2) IN (SELECT ref_0.c6 AS c_0
  FROM (t2 AS ref_1 FULL OUTER JOIN t1 AS ref_3
    ON (ref_1.vkey = ref_3.vkey)));
--- expected: the execution succeeds with correct result
--- actual result: the request times out after 30s 🐞
```

Figure 9: A bug found in ShardingSphere: a correlated subquery triggers per-row rescans, exhausting the connection pool and causing timeouts.

Example 5.1. Figure 8 shows a SQL query that triggers a logic bug in Citus. The query performs a `FULL OUTER JOIN` between a distributed table `t1`, sharded by `vkey` (32 shards by default), and a reference table `t2`. Executing the query should return a single row. Instead, Citus returns 32 identical rows, one per shard, indicating that the full outer join is executed independently on each shard, with the coordinator simply concatenating the results rather than properly aggregating them.

This bug stems from a flaw in the distributed planner that prevents Citus from recognizing certain joins, e.g., `FULL OUTER JOIN` with pseudo-constant restrictions. As a result, the outer join is not considered during planning, leading to an incorrect distributed execution plan. Citus treats this as a critical logic bug requiring user attention. In the current release, it explicitly raises an error instead of silently producing incorrect results.

Example 5.2. Figure 9 illustrates a query that exposes a timeout issue in ShardingSphere caused by connection pool exhaustion. The federation layer in ShardingSphere relies on SQL rewriting to retrieve data across data nodes, but it mishandles the correlated subquery. At runtime, for each outer row of `ref_0`, it rescans all shards of `ref_1` and `ref_3` without reusing intermediate results. Moreover, each scan acquires a new backend connection, quickly exhausting the connection pool and resulting in timeouts.

This bug is severe, as it can render the system unavailable: once the connection pool is saturated, subsequent requests may be delayed or fail while waiting for available connections. Moreover, it can be exploited by an attacker to launch denial-of-service attacks.

5.3 Comparison with State-of-the-Art

We compare DistRanger against two representative state-of-the-art DBMS testing tools, namely SQLancer [47, 53] and EET [32].[2]

We select SQLancer[3] as a strong baseline for *end-to-end* database testing due to its success in finding numerous bugs in production DBMSs over the years. Its current implementation supports testing Citus and ClickHouse, for which Ternary Logic Partitioning (TLP) [46] is used as the test oracle. TLP partitions a query into three derived queries whose results are combined and compared with the original query result; any mismatch indicates a bug.

EET is a recent testing tool that supports diverse queries by incorporating rich SQL features (e.g., subqueries and various join types). Given a query, EET rewrites selected expressions into semantically equivalent but structurally more complex forms, whose results are expected to be identical. Since EET also builds on DynSQL, its query generator is largely comparable to that of DistRanger, enabling a fair comparison of query executability (Section 5.3.2) and diversity (Section 5.3.3) in DDBMSs. Moreover, the shared codebase allows us to readily equip EET with our SDM to generate DDBMS instances, which it does not natively support. This enables comparisons between DistRanger and EET across all target DDBMSs.

5.3.1 Bug Detection. To examine how effective each tool is in bug detection, we conducted a 24-hour empirical analysis. The results are shown in Table 5. Overall, DistRanger identifies a total of 16 bugs, including 13 related to distribution mechanisms in DDBMSs, significantly outperforming SQLancer and EET. Notably, all bugs detected by EET are also found by DistRanger.

These performance gaps arise from fundamental design differences. Both SQLancer and EET primarily target centralized settings, which limits the quality of their generated queries in terms of (i)

[2]DistSQL [26] is a recent tester that targets logic bugs in DDBMSs. However, its codebase (approximately 28 KLOC) is not publicly available as of this writing, preventing a direct experimental comparison. See Section 6 for a detailed discussion.
[3]Specifically, we use the latest version of SQLancer as of April 27, 2026 [52].

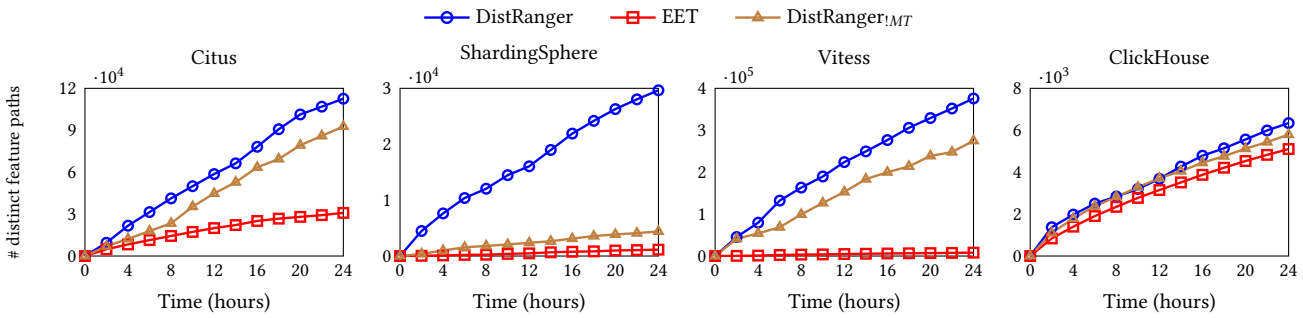

Figure 10: Comparison of query diversity, measured as the cumulative number of distinct feature paths extracted from the executable queries on each DDBMS over 24 hours.

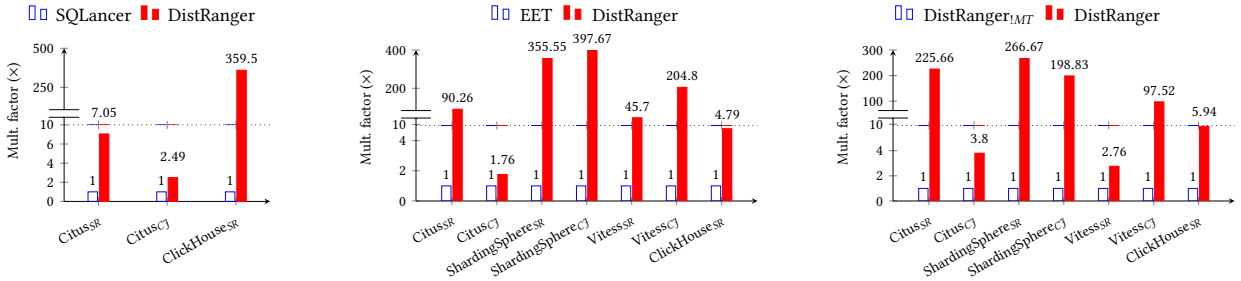

Figure 11: Frequency of triggering distributed query optimizations, shard routing (SR) and co-located joins (CJ), across four DDBMSs. For ClickHouse, we evaluate only SR, as it does not support CJ. Results are normalized, and the y-axis reports multiplicative factors relative to the baseline.

Table 6: Comparison of query executability rates over 24 hrs.

DDBMS	DistRanger	SQLancer	EET	DistRanger$_{!FB}$
Citus	87.6%	38.9%	25.3%	47.0%
ClickHouse	88.7%	33.0%	85.1%	84.5%
ShardingSphere	72.0%	–	3.1%	22.6%
Vitess	84.6%	–	13.8%	55.3%

executability, preventing a large fraction of them from being executed in DDBMSs, and (ii) diversity, hindering their ability to trigger diverse system behaviors. A detailed analysis is provided below.

Note that Table 3 (Section 5.2) reports a larger number of bugs, as our full testing campaign spanned several months.

5.3.2 Test Query Executability. Executability is crucial, as only executable queries can be used to exercise system behaviors and expose potential bugs. We measure the executability rate of queries generated by the three tools across the four DDBMSs. As shown in Table 6, DistRanger significantly outperforms EET (by up to 22.2×), as well as SQLancer (by up to 1.7× on the two systems it supports). This demonstrates the effectiveness of our feedback mechanism in guiding the generator toward queries accepted by DDBMSs.

Note that DistRanger's executability rate varies across DDBMSs. This is primarily due to differences in unsupported features across systems, as well as the limited time (24 hours) available for our feedback mechanism to learn and guide generation. Moreover, achieving a 100% executability rate is inherently challenging. One reason is that queries containing unsupported features may still be accepted by the DDBMS due to optimizer branch pruning, which can mask invalid substructures. We defer a detailed analysis to Section 5.4.2.

5.3.3 Test Query Diversity. The diversity of generated queries is another key factor for effective DDBMS testing. A tester that repeatedly generates structurally similar queries may exercise only a limited portion of the distributed query processing logic, even if they are executable. Therefore, we evaluate whether DistRanger can continuously explore diverse queries over time, compared to EET (as both tools share a common base query generator).

As described in Section 4.3, a feature path summarizes table-to-table structural relationships together with their effective distribution strategies. Accordingly, the number of unique feature paths serves as a proxy for the diversity of distributed execution patterns covered by a tool. We measure this using the cumulative number of distinct feature paths extracted from executable queries generated during 24 hours on each DDBMS. As shown in Figure 10, DistRanger consistently covers more unique feature paths than EET across all the DDBMSs. This demonstrates that DistRanger enables broader exploration of distributed query execution behaviors.

5.3.4 Triggering Specific Query Optimizations. Another advantage of DAT is its generality: it can be instantiated to target specific distributed query optimizations, such as shard routing and co-located joins (Section 4.2). Accordingly, we compare how frequently three tools trigger these two optimizations. For each tool, we generate 10K executable queries per run across five runs under randomly generated DDBMS instances. We instrument key entry points in the optimization logic to record when each optimization is triggered

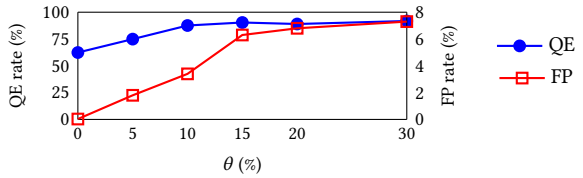

Figure 12: Query executability (QE) and false-positive (FP) rates under different values of θ.

(see the Appendix [68] for details); thus, trigger counts approximate its execution frequency. Since SQLancer does not support generating distributed tables for ClickHouse and thus cannot trigger the shard-routing logic, we randomly distribute the generated tables across the ClickHouse cluster to enable a meaningful comparison.

As shown in Figure 11 (left and middle), DistRanger triggers the two distributed query optimizations more frequently than both SQLancer and EET, achieving improvements of up to several hundred times. This is expected, as our mutation strategies, i.e., Δ_{PI} and Δ_{JA}, are explicitly designed to trigger these optimizations, while query generation in SQLancer and EET is largely agnostic to them.

5.4 A Closer Look At DistRanger

Having demonstrated DistRanger's effectiveness over state-of-the-art DBMS testing tools, we now analyze how each of its key design choices contributes to these gains.

5.4.1 Ablation Analysis. We examine two variants: (i) DistRanger without query mutation (i.e., using random query generation instead), denoted as DistRanger$_{!MT}$, to assess its effect on triggering diverse distributed query execution and optimizations; and (ii) DistRanger without the feedback mechanism, denoted as DistRanger$_{!FB}$, to evaluate its impact on the executability of generated queries.

Overall, both variants perform significantly worse on their respective metrics, as expected. In particular, as shown in Table 6, DistRanger$_{!FB}$ exhibits a lower query executability rate than DistRanger across the four DDBMSs, e.g., with up to a 2.2× reduction for ShardingSphere. Moreover, DistRanger$_{!MT}$ explores fewer distinct feature paths than DistRanger within the same time budget, as shown in Figure 10, with up to a 6× reduction for ShardingSphere.

In addition, we evaluate the effectiveness of our mutation strategies in targeting distributed query optimizations. Specifically, we compare how frequently DistRanger and DistRanger$_{!MT}$ trigger the two optimizations, namely shard routing and co-located joins. The experimental setup is the same as that used in Section 5.3.4. As shown in Figure 11 (right), DistRanger$_{!MT}$ triggers both optimizations far less frequently than DistRanger. For instance, the gap reaches approximately 190–250× on ShardingSphere.

Finally, we examine how the above degradations impact bug detection capability. As shown in Table 5, both variants find noticeably fewer bugs than DistRanger. This is expected: without the feedback mechanism, fewer queries can be successfully executed by the DDBMSs in the first place; without the query mutation, the probability of triggering diverse distributed execution behaviors is reduced, thereby lowering the likelihood of exposing bugs.

5.4.2 Threshold θ. In our approach, a feature path is rejected by the feedback mechanism when its historical query executability (QE) rate falls below θ (Section 4.3). Through empirical analysis (Figure 12), we observe that as θ increases, the overall QE rate improves and eventually stabilizes, while the false-positive (FP) rate, i.e., executable queries in the DDBMS incorrectly rejected by our mechanism, continues to increase, albeit at a decreasing rate.

In black-box testing, false positives are difficult, if not impossible, to avoid. Consider a simple example: a predicate such as `WHERE TRUE OR` *right* always evaluates to true. The optimizer may prune the *right* branch as a simplification, even if it contains unsupported structural features. As a result, the query can be executed by the DDBMS. When θ is too small (e.g., 0%–5%), branch pruning can delay learning, as the feedback mechanism may require many counterexamples before recognizing that a query is actually not executable in the system, resulting in a limited QE rate. When θ is too large (e.g., $\geq$ 20%), executable feature paths may be misclassified as unsupported by our mechanism, leading to higher FP rates and overly conservative mutations. Therefore, in practice, we use a moderate θ (e.g., 10%) to balance this trade-off.

6 Related Work

(D)DBMS Testing. Recent years have witnessed a surge of efforts on testing DBMSs; yet, issues arising from query processing and optimization in distributed settings remain largely underexplored.

SQLancer [47, 53] and EET [32] are state-of-the-art testing approaches that integrate both test generation (e.g., queries and database schemas) and test oracles. Despite their success in finding bugs in production DBMSs, they focus on centralized settings. In contrast, DAT is specifically designed for testing DDBMSs, targeting query-related issues in the distribution layer that are absent in centralized DBMSs. In particular, our experiments demonstrate that DAT outperforms both approaches in bug detection, with nearly all discovered bugs being specific to distributed query processing.

DistSQL [26] is a recent differential testing framework that targets logic bugs in DDBMSs. It compares the results of the same query executed on a DBMS configured in centralized and distributed modes. DAT distinguishes itself from DistSQL in three key aspects:

(1) While DAT currently uses the same oracle for identifying bugs, it is designed as a general testing framework. Additional test oracles can be naturally incorporated to enable the detection of a broader range of issues (see below and Section 7). In contrast, DistSQL tightly couples its logic bug detection with differential testing, relying on query plan discrepancies across deployment modes to generate queries that are more likely to trigger bugs.
(2) DAT emphasizes generating executable test queries, whereas DistSQL does not. Therefore, we conjecture that a non-trivial fraction of DistSQL-generated queries may be rejected by DDBMSs, thereby reducing overall testing efficacy.
(3) DAT enables targeted generation of queries that trigger specific distributed optimizations via SDM-guided mutation (i.e., μ), whereas DistSQL lacks such targeted exploration, potentially missing bugs related to these optimizations.

Nonetheless, DistSQL can expose certain classes of bugs not yet covered by DistRanger. First, its query generator supports SQL features not emphasized in our implementation (e.g., window functions with `PARTITION BY`), enabling it to generate relevant bug-triggering

queries. Second, it explores bugs related to system configurations, such as varying storage engines or enabling optional components.

These gaps reflect implementation choices rather than fundamental limitations of DAT. Missing SQL features can be incorporated into the base query generator without altering the core design, after which the mutation strategy can be extended and the feedback mechanism can continue to guide it using executability results. Configuration-level testing requires additional engineering effort, as noted in DistSQL. These extensions are left for future work.

There is also a substantial body of research on testing distributed systems. While some involves distributed databases, its focus is largely orthogonal, e.g., on synchronization failures [25] and recovery bugs [27]. In contrast, DAT focuses on issues arising from query processing. Exploring synergies between DAT and these techniques, e.g., fault injection [30, 48], is interesting future work, as it may, e.g., help trigger more diverse distributed execution behaviors.

From a technical perspective, our work is closely related to two lines of DBMS testing research, i.e., test generation and test oracles, which have thus far primarily focused on centralized settings.

Test Generation. A large body of prior work [8, 31, 36–38, 43, 54, 56] focuses on generating diverse test cases, including complex SQL queries and varied database schemas and configurations. DAT builds upon these advances, e.g., DynSQL [31]. Among these approaches, mutation-based techniques are often used to increase the diversity of queries, while feedback mechanisms are employed to guide test generation, aiming to improve test case quality (e.g., trigger a wider range of system behaviors).

However, queries that are syntactically valid and accepted by centralized DBMSs are not necessarily executable in DDBMSs due to distribution-specific constraints. Paradoxically, increasing query complexity, which has proven effective in triggering diverse system behaviors in centralized settings, can raise the likelihood of rejection in distributed settings. DAT addresses this dilemma by mutating queries based on the SDM and leveraging historical executability results to guide the mutation process.

Test Oracles. An orthogonal line of work is dedicated to the design of test oracles, which determine whether a test output matches the expected result. Representative oracles include (i) differential testing [43, 54], which compares outputs of identical queries executed under different configurations (e.g., centralized vs. distributed deployments of the same DBMS in our experiments); (ii) metamorphic testing [9, 32, 34, 46, 50, 51, 66], which verifies expected relations among the outputs of related inputs (e.g., SQLancer's and EET's test oracles described in Section 5.3); and (iii) crash and exception detection [31], achievable by simply inspecting system outputs.

DAT is designed as a general testing framework that currently focuses on test generation and already integrates the oracles in (i) and (iii) within DistRanger. Extending it to incorporate additional oracles, particularly those in (ii), is feasible, either directly or with minor adaptations. For instance, incorporating SQLancer's TLP or EET's semantically equivalent query transformations incurs no additional cost: for a SQL query and its derived query, we simply obtain an additional pair of execution results in the distributed setting; as a result, we compare the equivalence of four query results in total. However, designing test oracles that are effective specifically for DDBMSs remains an important direction for future work.

7 Discussion and Conclusion

We have presented DAT, a novel automated testing approach tailored to DDBMSs, together with its implementation, DistRanger. Using DistRanger, we have uncovered many previously unknown bugs in production DDBMSs and demonstrated its superior effectiveness in bug detection compared to the state-of-the-art. Our approach can be instantiated and extended to test a wide range of DDBMSs, as well as distributed query processing and optimization techniques (see below). This work complements prior efforts on centralized settings by addressing issues that arise specifically in the distribution layers of DDBMSs. Together, these efforts contribute to more reliable database systems.

Next, we discuss the limitations of this work and outline additional directions for future work beyond those covered in Section 6.

Limitations. While testing can find bugs, it does not guarantee their absence. DAT, like other bug detection approaches, shares this inherent limitation. Formally verifying DDBMSs could overcome this issue; yet, it remains extremely challenging, if not infeasible, for large-scale distributed systems [11, 28]. Moreover, DistRanger may produce duplicate bug reports, as bugs with a common root cause can be triggered by distinct test cases. We currently rely on manual deduplication; yet its automation (and also for root cause diagnosis) is an orthogonal and challenging research problem in system testing [13, 64]. Finally, DAT relies on explicit distributed strategy specifications, which are widely available in most DDBMSs. For systems with dynamic or user-invisible distribution strategies, inferring their specifications requires additional engineering effort.

Extension to Other Distributed Query Optimizations. We demonstrate DAT's effectiveness in triggering distributed query optimizations, primarily via shard routing and co-located joins. Nonetheless, DDBMSs support many other optimizations, which can be incorporated into DistRanger by defining Δ rules, along with the mapping ρ, that transform the query to satisfy optimization-specific structural preconditions. For example, testing *aggregation pushdown* [6, 17] (e.g., `GROUP BY` on the sharding key) can be enabled by replacing tables with sharded counterparts, rewriting the query to enforce the required aggregate and `GROUP BY` structure, and aligning grouping keys with sharding keys.

Discovering Other Types of Bugs. DAT can serve as a basis for discovering many other types of bugs. For example, it can be utilized to detect performance issues in DDBMSs, such as inefficient distributed query processing. A key challenge lies in the lack of effective test oracles, i.e., determining the expected execution time of a distributed query. Cardinality estimation [45] may provide a promising direction for addressing it.

In addition, despite recent advances in black-box checking of isolation levels [12, 29, 33, 39, 42, 55, 65, 67], e.g., serializability and weaker levels [10, 40], they are largely limited to key-value stores. DAT can be extended to validate isolation guarantees in *relational* DDBMSs by wrapping mutated SQL queries as transactions and leveraging their semantic interactions with diverse distribution strategies to explore rich concurrency behaviors.

Notably, all these applications build on a common foundation: query executability, a prerequisite for testing to proceed in DDBMSs, and test diversity, an enabler of diverse system behaviors, both of which lie at the core of our DAT approach.

References

[1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. 2006. *Compilers: Principles, Techniques, and Tools (2nd Edition)*. Addison-Wesley.

[2] Apache ShardingSphere. Accessed: April 2026. Alter Sharding Table Reference Rule. https://shardingsphere.apache.org/document/current/en/user-manual/shardingsphere-proxy/distsql/syntax/rdl/rule-definition/sharding/alter-sharding-table-reference-rule/.

[3] Apache ShardingSphere. Accessed: April 2026. Create Broadcast Table Rule. https://shardingsphere.apache.org/document/current/en/user-manual/shardingsphere-proxy/distsql/syntax/rdl/rule-definition/broadcast-table/create-broadcast-table-rule/.

[4] Apache ShardingSphere. Accessed: April 2026. Create Sharding Table Rule. https://shardingsphere.apache.org/document/current/en/user-manual/shardingsphere-proxy/distsql/syntax/rdl/rule-definition/sharding/create-sharding-table-rule/.

[5] Apache ShardingSphere. Accessed: April 2026. Load Single Table. https://shardingsphere.apache.org/document/current/en/user-manual/shardingsphere-proxy/distsql/syntax/rdl/rule-definition/single-table/load-single-table/.

[6] Apache ShardingSphere. Accessed: April 2026. Merger Engine: Group-by Merger. https://shardingsphere.apache.org/document/current/en/reference/sharding/merge.

[7] Apache ShardingSphere. Accessed: April 2026. Route Engine. https://shardingsphere.apache.org/document/current/en/reference/sharding/route/.

[8] Jinsheng Ba and Manuel Rigger. 2023. Testing Database Engines via Query Plan Guidance. In *Proceedings of the 45th International Conference on Software Engineering (ICSE '23)*. IEEE Press, 2060–2071. https://doi.org/10.1109/ICSE48619.2023.00174

[9] Jinsheng Ba and Manuel Rigger. 2024. Keep It Simple: Testing Databases via Differential Query Plans. *Proc. ACM Manag. Data* 2, 3, Article 188 (May 2024), 26 pages. https://doi.org/10.1145/3654991

[10] Hal Berenson, Phil Bernstein, Jim Gray, Jim Melton, Elizabeth O'Neil, and Patrick O'Neil. 1995. A critique of ANSI SQL isolation levels. In *Proceedings of the 1995 ACM SIGMOD International Conference on Management of Data (SIGMOD '95)*. ACM, 1–10. https://doi.org/10.1145/223784.223785

[11] James Bornholt, Rajeev Joshi, Vytautas Astrauskas, Brendan Cully, Bernhard Kragl, Seth Markle, Kyle Sauri, Drew Schleit, Grant Slatton, Serdar Tasiran, Jacob Van Geffen, and Andrew Warfield. 2021. Using Lightweight Formal Methods to Validate a Key-Value Storage Node in Amazon S3. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP '21)*. ACM, 836–850. https://doi.org/10.1145/3477132.3483540

[12] Zhiheng Cai, Si Liu, Hengfeng Wei, Yuxing Chen, and Anqun Pan. 2025. Fast Verification of Strong Database Isolation. *Proc. VLDB Endow.* 19, 4 (Dec. 2025), 563–575. https://doi.org/10.14778/3785297.3785300

[13] Yang Chen, Alex Groce, Chaoqiang Zhang, Weng-Keen Wong, Xiaoli Fern, Eric Eide, and John Regehr. 2013. Taming compiler fuzzers. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '13)*. ACM, 197–208. https://doi.org/10.1145/2491956.2462173

[14] Citus Data. Accessed: April 12, 2026. Citus Technical Documentation. https://github.com/citusdata/citus/blob/main/src/backend/distributed/README.md.

[15] Citus Data. Accessed: April 12, 2026. Concepts: Co-Location. https://docs.citusdata.com/en/stable/get_started/concepts.html#co-location.

[16] Citus Data. Accessed: April 12, 2026. Concepts: Table Types. https://docs.citusdata.com/en/stable/get_started/concepts.html#table-types.

[17] Citus Data. Accessed: April 12, 2026. Querying Distributed Tables (SQL): Aggregate Functions. https://docs.citusdata.com/en/latest/develop/reference_sql.html.

[18] Citus Data. Accessed: April 12, 2026. SQL Support and Workarounds. https://docs.citusdata.com/en/stable/develop/reference_workarounds.html.

[19] ClickHouse. Accessed: April 12, 2026. Distributed table engine. https://clickhouse.com/docs/engines/table-engines/special/distributed.

[20] ClickHouse. Accessed: April 12, 2026. MergeTree table engine. https://clickhouse.com/docs/engines/table-engines/mergetree-family/mergetree.

[21] ClickHouse. Accessed: April 12, 2026. Replicated table engines. https://clickhouse.com/docs/engines/table-engines/mergetree-family/replication.

[22] ClickHouse. Accessed: Feb. 13, 2026. Fast Open-Source OLAP DBMS. https://clickhouse.com/.

[23] George Coulouris, Jean Dollimore, Tim Kindberg, and Gordon Blair. 2011. *Distributed Systems: Concepts and Design*. Addison-Wesley Publishing Company.

[24] Umur Cubukcu, Ozgun Erdogan, Sumedh Pathak, Sudhakar Sannakkayala, and Marco Slot. 2021. Citus: Distributed PostgreSQL for Data-Intensive Applications. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21)*. ACM, 2490–2502. https://doi.org/10.1145/3448016.3457551

[25] Cundi Fang, Jie Liang, Zhiyong Wu, Jingzhou Fu, Zhouyang Jia, Chun Huang, Yu Jiang, and Shanshan Li. 2025. DepState: Detecting Synchronization Failure Bugs in Distributed Database Management Systems. *Proc. ACM Softw. Eng.* 2, ISSTA, Article ISSTA088 (June 2025), 22 pages. https://doi.org/10.1145/3728965

[26] Zi-Xuan Fu, Jia-Ju Bai, Hong-Bo Feng, and Kang Chen. 2026. One DBMS, Two Modes, and a Bunch of Bugs: Catching Logic Bugs in Distributed DBMSs via Differential Testing. *Proc. ACM Manag. Data* 4, 1, Article 59 (April 2026), 24 pages. https://doi.org/10.1145/3786673

[27] Yu Gao, Wensheng Dou, Dong Wang, Wenhan Feng, Jun Wei, Hua Zhong, and Tao Huang. 2023. Coverage Guided Fault Injection for Cloud Systems. In *Proceedings of the 45th International Conference on Software Engineering (ICSE '23)*. IEEE Press, 2211–2223. https://doi.org/10.1109/ICSE48619.2023.00186

[28] Heidi Howard, Markus A. Kuppe, Edward Ashton, Amaury Chamayou, and Natacha Crooks. 2025. Smart Casual Verification of the Confidential Consortium Framework. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. USENIX Association, 259–276. https://www.usenix.org/conference/nsdi25/presentation/howard

[29] Kaile Huang, Si Liu, Zhenge Chen, Hengfeng Wei, David Basin, Haixiang Li, and Anqun Pan. 2023. Efficient Black-Box Checking of Snapshot Isolation in Databases. *Proc. VLDB Endow.* 16, 6 (Feb. 2023), 1264–1276. https://doi.org/10.14778/3583140.3583145

[30] Jepsen. Accessed: April 1, 2026. Distributed Systems Safety Research. http://jepsen.io/.

[31] Zu-Ming Jiang, Jia-Ju Bai, and Zhendong Su. 2023. DynSQL: Stateful Fuzzing for Database Management Systems with Complex and Valid SQL Query Generation. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, 4949–4965. https://www.usenix.org/conference/usenixsecurity23/presentation/jiang-zu-ming

[32] Zu-Ming Jiang and Zhendong Su. 2024. Detecting Logic Bugs in Database Engines via Equivalent Expression Transformation. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, 821–835. https://www.usenix.org/conference/osdi24/presentation/jiang

[33] Kyle Kingsbury and Peter Alvaro. 2020. Elle: inferring isolation anomalies from experimental observations. *Proc. VLDB Endow.* 14, 3 (Nov. 2020), 268–280. https://doi.org/10.14778/3430915.3430918

[34] Jinhui Lai, Chi Zhang, Bingyan Li, Chenglin Liang, Jie Liang, Zhiyong Wu, Jingzhou Fu, Yu Jiang, and Zichen Xu. 2025. SRS: Detecting Logic Bugs of Join Implementation in DBMSs via Set Relation Synthesis. *Proc. ACM Manag. Data* 3, 6, Article 363 (Dec. 2025), 24 pages. https://doi.org/10.1145/3769828

[35] Ruiyuan Li, Liang Zhang, Juan Pan, Junwen Liu, Peng Wang, Nianjun Sun, Shanmin Wang, Chao Chen, Fuqiang Gu, and Songtao Guo. 2022. Apache ShardingSphere: A Holistic and Pluggable Platform for Data Sharding. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. 2468–2480. https://doi.org/10.1109/ICDE53745.2022.00230

[36] Jie Liang, Yaoguang Chen, Zhiyong Wu, Jingzhou Fu, Mingzhe Wang, Yu Jiang, Xiangdong Huang, Ting Chen, Jiashui Wang, and Jiajia Li. 2023. Sequence-Oriented DBMS Fuzzing. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. 668–681. https://doi.org/10.1109/ICDE55515.2023.00057

[37] Jie Liang, Zhiyong Wu, Jingzhou Fu, Mingzhe Wang, Chengnian Sun, and Yu Jiang. 2024. Mozi: Discovering DBMS Bugs via Configuration-Based Equivalent Transformation. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. ACM, Article 135, 12 pages. https://doi.org/10.1145/3597503.3639112

[38] Yu Liang, Song Liu, and Hong Hu. 2022. Detecting Logical Bugs of DBMS with Coverage-based Guidance. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, 4309–4326. https://www.usenix.org/conference/usenixsecurity22/presentation/liang

[39] Si Liu, Long Gu, Hengfeng Wei, and David Basin. 2024. Plume: Efficient and Complete Black-Box Checking of Weak Isolation Levels. *Proc. ACM Program. Lang.* 8, OOPSLA2, Article 302 (Oct. 2024), 29 pages. https://doi.org/10.1145/3689742

[40] Si Liu, Luca Multazzu, Hengfeng Wei, and David A. Basin. 2024. NOC-NOC: Towards Performance-optimal Distributed Transactions. *Proc. ACM Manag. Data* 2, 1, Article 9 (March 2024), 25 pages. https://doi.org/10.1145/3639264

[41] Microsoft Learn. Accessed: April 3, 2026. Citus for PostgreSQL: Performance Tuning. https://learn.microsoft.com/en-us/postgresql/citus/performance-tuning?view=citus-14. Section: Scale-out performance.

[42] Lasse Møldrup and Andreas Pavlogiannis. 2025. AWDIT: An Optimal Weak Database Isolation Tester. *Proc. ACM Program. Lang.* 9, PLDI, Article 209 (June 2025), 25 pages. https://doi.org/10.1145/3742465

[43] PingCAP. Accessed: April 2026. Go-Randgen. https://github.com/pingcap/go-randgen.

[44] PingCAP. Accessed: March 11, 2026. TiDB Documentation: TiFlash Overview. https://docs.pingcap.com/tidb/stable/tiflash-overview/

[45] Manuel Rigger and Zhendong Su. 2020. Detecting optimization bugs in database engines via non-optimizing reference engine construction. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2020)*. ACM, 1140–1152. https://doi.org/10.1145/3368089.3409710

[46] Manuel Rigger and Zhendong Su. 2020. Finding bugs in database systems via query partitioning. *Proc. ACM Program. Lang.* 4, OOPSLA, Article 211 (Nov. 2020), 30 pages. https://doi.org/10.1145/3428279

[47] Manuel Rigger and Zhendong Su. 2020. Testing Database Engines via Pivoted Query Synthesis. In *14th USENIX Symposium on Operating Systems Design and*

Implementation (OSDI 20). USENIX Association, 667–682. https://www.usenix.org/conference/osdi20/presentation/rigger

[48] AWS Cloud Services. Accessed: April 2026. AWS Fault Injection Service. https://aws.amazon.com/fis/.

[49] Avi Silberschatz, Henry F. Korth, and S. Sudarshan. 2020. *Database System Concepts, Seventh Edition*. McGraw-Hill Book Company.

[50] Jiansen Song, Wensheng Dou, Yu Gao, Ziyu Cui, Yingying Zheng, Dong Wang, Wei Wang, Jun Wei, and Tao Huang. 2024. Detecting Metadata-Related Logic Bugs in Database Systems via Raw Database Construction. *Proc. VLDB Endow.* 17, 8 (April 2024), 1884–1897. https://doi.org/10.14778/3659437.3659445

[51] Jiansen Song, Wensheng Dou, Yingying Zheng, Yu Gao, Ziyu Cui, Wei Wang, and Jun Wei. 2025. Detecting Schema-Related Logic Bugs in Relational DBMSs via Equivalent Database Construction. *Proc. VLDB Endow.* 18, 7 (March 2025), 2281–2294. https://doi.org/10.14778/3734839.3734861

[52] SQLancer. Accessed: April 27, 2026. Commit d38ddcb. https://github.com/sqlancer/sqlancer/tree/d38ddcb.

[53] SQLancer. Accessed: Nov. 2025. SQLancer. https://github.com/sqlancer/sqlancer.

[54] SQLsmith. Accessed: Nov. 2025. SQLsmith. https://github.com/anse1/sqlsmith.

[55] Cheng Tan, Changgeng Zhao, Shuai Mu, and Michael Walfish. 2020. Cobra: Making Transactional Key-Value Stores Verifiably Serializable. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 63–80. https://www.usenix.org/conference/osdi20/presentation/tan

[56] Xiu Tang, Sai Wu, Dongxiang Zhang, Feifei Li, and Gang Chen. 2023. Detecting Logic Bugs of Join Optimizations in DBMS. *Proc. ACM Manag. Data* 1, 1, Article 55 (May 2023), 26 pages. https://doi.org/10.1145/3588909

[57] Vitess. Accessed: April 12, 2026. Lookup as Primary Vindex. https://vitess.io/docs/24.0/user-guides/vschema-guide/lookup-as-primary/.

[58] Vitess. Accessed: April 12, 2026. MySQL Compatibility: SQL Support. https://vitess.io/docs/24.0/reference/compatibility/mysql-compatibility/#sql-support.

[59] Vitess. Accessed: April 12, 2026. Reference Tables. https://vitess.io/docs/24.0/reference/vreplication/reference_tables/.

[60] Vitess. Accessed: April 12, 2026. Sharded Keyspace. https://vitess.io/docs/24.0/user-guides/vschema-guide/sharded/.

[61] Vitess. Accessed: April 12, 2026. Unsharded Keyspace. https://vitess.io/docs/24.0/user-guides/vschema-guide/unsharded/.

[62] Vitess. Accessed: April 2, 2026. Query Plans Classification. https://vitess.io/docs/24.0/reference/query-serving/metrics.

[63] Vitess. Accessed: Feb. 13, 2026. Scalable. Reliable. MySQL-compatible. Cloud-native. https://vitess.io/.

[64] Qianqian Wang, Chris Parnin, and Alessandro Orso. 2015. Evaluating the usefulness of IR-based fault localization techniques. In *Proceedings of the 2015 International Symposium on Software Testing and Analysis (ISSTA 2015)*. ACM, 1–11. https://doi.org/10.1145/2771783.2771797

[65] Hengfeng Wei, Jiang Xiao, Na Yang, Si Liu, Zijing Yin, Yuxing Chen, and Anqun Pan. 2025. Boosting End-to-End Database Isolation Checking via Mini-Transactions. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. 3998–4010. https://doi.org/10.1109/ICDE65448.2025.00298

[66] Chi Zhang and Manuel Rigger. 2025. Constant Optimization Driven Database System Testing. *Proc. ACM Manag. Data* 3, 1, Article 24 (Feb. 2025), 24 pages. https://doi.org/10.1145/3709674

[67] Jian Zhang, Ye Ji, Shuai Mu, and Cheng Tan. 2023. Viper: A Fast Snapshot Isolation Checker. In *Proceedings of the Eighteenth European Conference on Computer Systems (EuroSys '23)*. ACM, 654–671. https://doi.org/10.1145/3552326.3567492

[68] Zhou Zhou, Si Liu, Hengfeng Wei, and Min Zhang. 2026. *Distribution-Aware Distributed Database Testing*. Technical Report. https://github.com/duerwuyi/DAT/tech-rpt.pdf.

A Appendix

This section provides additional details to complement the experimental results in Section 5. It consists of two parts:

- A complete bug list, including detailed descriptions, bug types, status, and SQL features involved in bug-triggering queries, as shown in Table 8. This corresponds to Section 5.2.
- The key entry points instrumented in the codebase of each DDBMS for measuring optimization triggering (see below). This corresponds to Section 5.3.4.

Table 7: Instrumentation targets for measuring optimization triggering. Each entry denotes a specific function or source file serving as a measurement target.

DDBMS	Opt.	Instrumented entry point
Citus	SR	`shard_pruning.c`
Citus	CJ	`query_colocation_checker.c`
ShardingSphere	SR	`routeByShardingConditions`
ShardingSphere	CJ	`routeByShardingConditionsWithCondition`
Vitess	SR	`RoutingParameters.equal`
Vitess	CJ	`joinMerging.mergeShardedRouting`
ClickHouse	SR	`optimize_skip_unused_shards`

To measure how frequently generated queries trigger distributed query optimizations, we instrument entry points for the distributed query planning or routing logic in the codebase. Table 7 summarizes the instrumentation targets for the four DDBMSs. Since different systems expose shard routing and co-located joins through different internal modules, the instrumentation is system-specific.

For Citus, we instrument its distributed planner. Specifically, we use `shard_pruning.c` to measure shard-routing behavior because this file implements the shard routing (or pruning) logic that determines whether only a subset of shards needs to be accessed. For co-located joins, we instrument `query_colocation_checker.c`, which checks whether distributed tables satisfy the co-location requirements needed for co-located join execution.

For ShardingSphere, we instrument the Java routing layer and record whether execution reaches the corresponding route-engine logic inside the Java virtual machine. We use `routeByShardingConditions` to measure shard routing, since this component routes a query based on extracted sharding conditions. For co-located joins, we use `routeByShardingConditionsWithCondition`, which additionally handles routing decisions involving co-located joins.

For Vitess, we leverage its built-in query planning and routing components. We instrument `RoutingParameters.equal` to find when Vitess recognizes shard routing. For co-located joins, we instrument `joinMerging.mergeShardedRouting`, which is invoked when Vitess attempts to perform co-located joins.

For ClickHouse, we evaluate `optimize_skip_unused_shards` by comparing query execution with this option enabled and disabled. A query is counted as triggering shard routing when enabling this option changes the internal execution path or query plan, indicating that ClickHouse can skip irrelevant shards by shard routing. ClickHouse does not expose the co-located join optimization.

Table 8: A complete list of bugs detected by DistRanger.

	DDBMS	Description	Type	Status	Features Involved
1	Citus	A `FULL OUTER JOIN` between a distributed table and a reference table produce duplicate result due to no access to *set_join_pathlist_hook*.	logic	fixed	join
2	Citus	An `UPDATE` statement with distributed table produce wrong result due to wrong subplan of `WHERE` filter.	logic	fixed	DML, subquery
3	Citus	Subquery with reference table returns wrong result due to wrong single-shard routing logic.	logic	fixed	DML
4	Citus	Adding `WHERE TRUE` in subquery flips result due to wrong distributed query planning.	logic	fixed	subquery
5	Citus	Server crashes when re-adding a node during reset.	crash	investigating	DQL
6	Citus	A `Delete` statement crashes due to an assertion failure in distributed query rewriting.	crash	fixed	DML, subquery
7	Citus	Subquery with `LIMIT` returns wrong result due to wrong distributed subplan.	logic	fixed	subquery
8	Citus	Segmentation fault occurs on `LEFT JOIN` to a distributed table and correlated subqueries.	crash	investigating	subquery, join
9	ShardingSphere	Wrong result of `information_schema.tables` when sharding a table.	logic	investigating	metadata
10	ShardingSphere	Incorrect result window aggregate on sharded table due to wrong aggregation logic.	logic	confirmed	window
11	ShardingSphere	Correlated subquery in `HAVING` cannot reference outer query column in SQL parser.	error	fixed	subquery, group by
12	ShardingSphere	Exception when `SELECT` version() due to wrong function verify and routing.	exception	confirmed	/
13	ShardingSphere	Timestamp output adds trailing ".0" in SQL format.	logic	confirmed	/
14	ShardingSphere	Correlated subquery triggers timeout due to wrong distributed query rewriting.	timeout	investigating	subquery, join
15	ShardingSphere	Correlated `IN`-subquery fails in a non-join subquery due to wrong distributed planing logic.	error	fixed	subquery, join
16	ShardingSphere	Co-located join inside scalar subquery fails due to wrong distributed query rewriting.	error	fixed	join, subquery
17	ShardingSphere	Result of `SELECT DISTINCT` duplicates due to wrong distributed query rewriting.	logic	fixed	distinct
18	ClickHouse	Nested `EXISTS` with deep correlated reference fails.	exception	investigating	subquery
19	ClickHouse	Result of `GLOBAL RIGHT OUTER JOIN` on distributed tables duplicates.	logic	investigating	join
20	Vitess	ApplyVSchema accepts empty auto_increment object and later `INSERT` fails.	error	investigating	shard
21	Vitess	Cannot insert into pinned table, got no primary vindex due to wrong distributed query routing.	error	confirmed	shard
22	Vitess	Window function without single-shard restriction returns wrong result with uncorrelated scalar subquery.	logic	investigating	subquery, window
23	Vitess	VTGate panic when `WHERE` contains `LTRIM` on sharded tables.	crash	investigating	subquery, join
24	Vitess	VTGate planning appears to hang on queries with many uncorrelated scalar subqueries.	timeout	investigating	subquery, join
25	Vitess	subquery is incorrectly dropped on sharded keyspace.	logic	investigating	subquery
26	Vitess	Unknown column in `WHERE` clause of scalar subquery.	error	investigating	subquery, aggregation
27	Vitess	Planner drops constant-true `OR` branch and returns wrong result on sharded keyspace.	logic	investigating	subquery

Continued on next page

Table 8: A complete list of bugs detected by DistRanger (Continued)

	DDBMS	Description	Type	Status	Features Involved
28	Vitess	VTGate panic in planner when comparing two scalar subqueries.	crash	confirmed	subquery
29	Vitess	Reference table `SELECT` incorrectly rewritten to `WHERE` 1!=1 with nested `EXISTS`.	logic	investigating	subquery
30	Vitess	Reference table `OUTER JOIN` on preserved side duplicates rows across shards.	logic	investigating	join
31	Vitess	Co-located join rewrite breaks `LEFT OUTER JOIN` semantics due to wrong distributed query rewriting.	logic	confirmed	join